
DS M1104 Partie 1

Documents autorisés : **Aide-mémoire distribué ce jour**

Durée : **1 heure (les deux parties)**

Exercice 1 Schémas et requêtes - SQL

Une réflexion portant sur la création d'une association d'amateurs de bières a donné naissance aux tables décrites en annexe qui ont été implantées dans une base de données PostgreSQL.

Un buveur et une bière sont identifiés par un identifiant *ad hoc*¹.

La présence d'un couple dans la relation **aimer** indique qu'une bière est appréciée d'un buveur (ou que ce buveur apprécie cette bière).

Un buveur peut ne pas apprécier de bières, ou en apprécier plusieurs.

Une bière peut n'être appréciée d'aucun buveur, ou de plusieurs buveurs.

Question

Pour chacune des tables définies, proposez la commande permettant d'ajouter dans celle-ci la contrainte de clé primaire qui convient.

1. Table **buveur**

2. Table **biere**

3. Table **aimer**

On s'intéresse maintenant aux bières appréciées par les buveurs.

4. Proposez une commande permettant d'ajouter le fait que le buveur n°1 aime la bière n°4.

5. Proposez une commande permettant de dire que le buveur n°6 n'aime plus la bière n°3, mais la bière n°7.

1. créé dans ce but.

6. La modification précédente est-elle possible dans l'état actuelle de la base (schéma et valeur)? Pourquoi²?

7. Proposez une commande permettant de palier au problème rencontré dans la question précédente.

8. En considérant que la commande précédente a bien été exécutée et validée, proposez une séquence permettant de supprimer toutes les tables. La séquence doit pouvoir s'exécuter sans erreur dans l'ordre des commandes que vous proposerez.

On voudrait maintenant interroger la base de données. Proposez des requêtes en SQL permettant de répondre aux questions qui suivent.

9. L'ensemble des noms des buveurs qui apprécient des bières de type 'pils'.

10. Pour chaque buveur quel qu'il soit, caractérisé par son numéro, son nom et son prénom, le nombre de bières qu'il apprécie.

2. L'absence de raison entrainera un 0 à la question.

DS M1104 Partie 2

Documents autorisés : **Aide-mémoire distribué ce jour**

Durée : **1 heure (les deux parties)**

Exercice 1 Requêtes - Tutorial D

A partir de la bases de données décrite en annexe, rédigez **en Tutorial D** les requêtes suivantes. Les requêtes rédigées dans un autre langage seront comptés comme fausses.

1. Les identifiants et noms des buveurs ?

2. L'ensemble des noms des buveurs qui apprécient des bières.

3. L'ensemble des noms des bières qu'aucun buveur n'apprécie.

4. L'ensemble des numéros des buveurs qui n'aiment ni les bières 'pils' ni les bières 'aromatisée'.

5. Pour chaque buveur appréciant des bières, la moyenne de degré d'alcool des bières qu'il apprécie.

6. L'ensemble des numéros et noms des buveurs qui apprécient au moins 2 bières de type 'pils', ainsi que le nombre de bières de type 'pils' appréciées pour chacun d'eux.

7. L'ensemble des couples de numéros de buveurs qui ont au moins une bière qu'ils apprécient en commun.

Aide mémoire Tutorial D

LMD (Langage de Manipulation des Données)

■ UNION

```
s = r1 UNION r2
```

- **s**, **r1** et **r2** sont des relations de même type
- la valeur de **s** est l'ensemble des n-uplets, où chacun d'entre eux est soit un n-uplet de **r1**, soit un n-uplet de **r2** (ou des deux).

■ INTERSECTION

```
s = r1 INTERSECT r2
```

- **s**, **r1** et **r2** sont des relations de même type
- la valeur de **s** est l'ensemble des n-uplets, où chacun d'entre eux est un n-uplet de **r1** et de **r2**.

■ DIFFERENCE

```
s = r1 MINUS r2
```

- **s**, **r1** et **r2** des relations de même type
- la valeur de **s** est l'ensemble des n-uplets, où chacun d'entre eux est un n-uplet de **r1** et n'est pas un n-uplet de **r2**.

■ RESTRICTION

```
s = r WHERE c
```

- **s** et **r** sont de même type
- **c** est une condition, expression possiblement vraie sur un ou plusieurs n-uplet de **r**
- la valeur de **s** est l'ensemble des n-uplets de **r** qui vérifient (chacun) la condition **c**.

■ PROJECTION

```
s = r {a1, a2, ..., an}
  = r {ALL BUT b1, ..., bm}
```

- {**a1**, **a2**, ..., **an**} et {**b1**, ..., **bm**} sont des sous-ensembles disjoints de l'ensemble des attributs de **r**
- l'entête de **s** est le sous-ensemble de l'entête de **r** donné par {**a1**, **a2**, ..., **an**}
- la valeur de **s** est l'ensemble des n-uplets de **r** dont on retire les attributs de {**b1**, ..., **bm**} (ou dont on n'affiche que les attributs de {**a1**, **a2**, ..., **an**}).

■ INSERTION : (exemples)

```
INSERT produit RELATION {
  TUPLE { no_prod 'P8',
    qte_stock 150, categ 's' } };
ou par affectation
produit := produit UNION RELATION{TUPLE{...}, ...}
```

■ JOINTURE

```
s = r1 join r2
```

- **r1** et **r2** sont des relations dont les types ont une intersection non-nulle (un attribut ou plusieurs en commun : même non, même type).
- le type de **s** est l'union des types de **r1** et de **r2**.
- la valeur de **s** est l'ensemble des n-uplets de **r1** combinés avec l'ensemble des n-uplets de **r2** qui ont une valeur qui correspond sur l'ensemble des attributs en commun.
- on appelle l'égalité des attributs en commun la condition de jointure

■ AGREGATS

```
EXTEND r : {x:=f1, y:=f2}
```

- où **f1** et **f2** sont des formules arbitraires, basées par exemple sur des fonctions d'agrégat
- permet d'étendre l'entête d'une relation avec ces formules.

SUMMARIZE r BY {b1,...,bm}:

```
{x:=f1, ..., z:=fn} où
```

- **r** est une relation
- **b1**, ..., **bm** sont des noms d'attributs de **r**, sur lesquels on construit les regroupements (dans l'ordre de leur énumération)
- **f1**, ..., **fn** sont des formules arbitraires impliquant des fonctions d'agrégat

Les fonctions d'agrégats peuvent être AVG, SUM, COUNT, MAX, MIN, *etc.*

Une commande bien utile pour déclarer des variables relationnelles intermédiaires (et locales) :

```
WITH (pv := produit
  WHERE couleur = DCOULEUR('Vert'),
  ftpv := fourniture JOIN pv) :
SUMMARIZE ftpv BY {nofour} :
{produit := count()}
```

■ SUPPRESSION : (exemple)

```
DELETE produit
WHERE no_prod = 'P1';
```

LDD (Langage de Description des données)

■ Création de relation :

```
VAR nom_relation BASE RELATION{
  nom_attr1 type_attr1,
  nom_attr2 type_attr2, ... }
KEY {liste_attr_dans_clé1}
KEY {liste_attr_dans_clé2};
```

■ Clé étrangère :

```
CONSTRAINT relation_référencante_fk1
  relation_référencante{liste_attributs} <=
  relation_référencée{liste_attributs}
```

■ Suppression :

```
DROP VAR nom_relation;
```

Quand il s'agit d'un type, remplace **VAR** par **TYPE**.
De même, pour **CONSTRAINT**.

LMD SQL PostgreSQL

```
SELECT [ALL | DISTINCT] < projection>
FROM <objets>
[WHERE <clause de restriction> ]
[GROUP BY <clause de groupage> [HAVING <condition>]]
[ORDER BY <clause d'ordonnancement>] ;
```

PRODUIT CARTESIEN : utilisation dans la clause FROM

- <nom_table1> [alias] CROSS JOIN <nom_table2> [alias]

JOINTURE: utilisation dans la clause FROM de l'opérateur JOIN avec l'une des syntaxes suivantes :

- <nom_table1> [alias] INNER JOIN <nom_table2> [alias] ON <condition> ou
- <nom_table1> [alias] {LEFT | RIGHT | FULL} OUTER JOIN <nom_table2> [alias] ON <condition>
- <nom_table1> [alias] NATURAL JOIN <nom_table2> [alias]

OPERATEURS ENSEMBLISTES : UNION | INTERSECT | EXCEPT entre deux requêtes SELECT

INSERT :

INSERT INTO <nom_table> [(<liste de nom_colonne>)] VALUES(<liste de valeurs>) ;

INSERT INTO <nom_table> [(<liste de nom_colonne>)] requête ;

DELETE : destruction de n-uplets dans **une** table

DELETE [FROM] <nom_table> : efface tous les n-uplets de la table.

DELETE [FROM] <nom_table> WHERE <condition> : efface tous les n-uplets sélectionnés par la condition.

UPDATE : permet de modifier la valeur de colonnes de n-uplets d'**une** table.

UPDATE <table> SET nom_colonne = <expression> | (requête) [WHERE <condition>] ;

LDD LDC SQL PostgreSQL

Création d'un schéma de table

CREATE TABLE <nom_table> (<déclaration colonne>[, {<déclaration colonne> | <contrainte table>}]");

<déclaration colonne> : <nom-colonne> <type> [<contrainte colonne> [<contrainte colonne>]]"

Principaux types de données

<type> : CHAR(n) | DATE | NUMERIC[(n,p)] | VARCHAR(n)

<contrainte colonne> : CONSTRAINT <nom_contrainte>

{PRIMARY KEY | [NOT] NULL |
UNIQUE | REFERENCES <nom_table> [(<nom_colonne>)] |
CHECK (<condition>)}

<contrainte table> : CONSTRAINT <nom_contrainte>

{ PRIMARY KEY(<nom_colonne>[, <nom_colonne>]")
| [NOT] NULL |
UNIQUE (<nom_colonne>[, <nom_colonne>]") |
FOREIGN KEY (<nom_colonne>[, <nom_colonne>]")
REFERENCES <nom_table> [(<nom_colonne>[, <nom_colonne>]") |
CHECK (<condition>) }

Création avec peuplement: CREATE est suivi d'une requête dont le résultat est chargé dans la table déclarée

CREATE TABLE <nom_table> (...) AS (requête).

Destruction d'un schéma : DROP TABLE <nom_table> (détruit le contenu éventuel **et** le schéma de la table).

Modification d'un schéma :

ALTER TABLE <nom_table> ADD (déclaration de colonne ou de contrainte) ;

ALTER TABLE <nom_table> DROP (colonne ou contrainte)

ALTER TABLE <nom_table> ALTER COLUMN (nom_de colonne , avec nouveau type éventuel et/ou une nouvelle Contrainte).

DS M1104 - Annexe

Ce schéma relationnel exprimé en SQL concerne la gestion d'amateurs de bières¹. Il décrit des ensembles de buveurs (**buveur**) et de bières(**biere**), ainsi que l'association entre les bières et les buveurs (**aimer**) qui représente quelles bières sont appréciées par quels buveurs.

```
CREATE TABLE buveur (  
    idbuveur INTEGER NOT NULL,  
    nomb VARCHAR(30) NOT NULL,  
    prenomb VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE aimer (  
    idbuveur INTEGER NOT NULL,  
    idbiere INTEGER NOT NULL);
```

```
CREATE TABLE biere (  
    idbiere INTEGER NOT NULL,  
    nombiere VARCHAR(30) NOT NULL,  
    degre NUMERIC(5,2) NOT NULL,  
    typebiere VARCHAR(20) NOT NULL);
```

La valeur de la base est la suivante :

buveur		
idbuveur	nomb	prenomb
1	'Eastwood'	'Clint'
2	'Abitbol'	'Georges'
3	'DeNice'	'Brice'
4	'Bricot'	'Juda'
5	'Meurdesoif'	'Jean'
6	'Kollyck'	'Al'

biere			
idbiere	nombiere	degre	typebiere
1	Kronembourg	4	pils
2	Pilsener	3	pils
3	Heinecken	4	pils
4	Leffe	6	triple
5	Smirnoff	4	aromatisee
6	Dremmwel	6	ale

aimer	
idbuveur	idbiere
1	2
1	3
2	3
3	1
3	2
3	3
3	4
3	5
3	6
4	1
6	1
6	3
6	5

1. A consommer avec modération. L'abus d'alcool est dangereux pour la santé.